



EASYIMMO

Rapport Final

Table des matières

Table des matières	2
1. Définition des besoins	3
2. Identification des éléments clés	4
Acteurs et intervenants	4
Fonctionnalités principales	4
Entités de données principales	4
3. Modèle entité-association	5
4. Schéma physique de la base de données	6
5. Prototypes des écrans	8
6. Bases de la solution .NET	10
Architecture en couches	10
Pattern MVVM	10
Entity Framework	10
7. Structure de la solution .NET	11
Organisation des projets	11
ImmoApp.DataAccess (Modèles)	11
ImmoApp.BLL (Services)	11
ImmoAppUI (ViewModels)	12
ImmoAppUI (Views)	12
8. Script SQL de création de la base de données	13
9. Présentation de l'application	28
Dashboard	28
Page Biens	28
Détail d'un bien	29
Calendrier	29
Documentations de l'application	30
Gestion des clients	30
Gestion des agents	31
10. Conclusion	32
Annexes	32
Document d'instructions Configuration	32
Prérequis	32
Etapas de configuration	32

1. Définition des besoins

Une petite agence immobilière a fait appel à nos services afin de digitaliser la gestion quotidienne de ses activités. Jusqu'à présent, l'agence utilisait des fichiers Excel et des documents papier, une méthode qui atteignait ses limites face à la croissance de l'entreprise.

L'application EasyImmo a été développée pour répondre aux besoins suivants :

- Suivre les biens dont l'agence s'occupe (vente, location ou autre)
- Gérer les différents statuts d'un bien au cours du temps avec un historique complet
- Accompagner et suivre les personnes qui interagissent avec l'agence
- Visualiser et gérer les événements importants liés à un bien
- Retrouver rapidement l'information utile : biens disponibles, visites effectuées, accords conclus

L'application desktop EasyImmo a été développée en C# avec le framework .NET 8 et MAUI pour l'interface utilisateur. La base de données relationnelle est implémentée sous SQL Server et l'accès aux données est géré via Entity Framework Core.

Changements par rapport à l'analyse préliminaire : certaines fonctionnalités initialement prévues (intégration API, Google Maps, rapports KPI, envoi automatique de documents) ont été abandonnées pour se concentrer sur les fonctionnalités essentielles demandées dans l'énoncé.

2. Identification des éléments clés

Acteurs et intervenants

- Les agents de l'agence : utilisateurs principaux de l'application
- Les clients : acheteurs, locataires, propriétaires, partenaires
- Le responsable de l'agence : supervise les performances

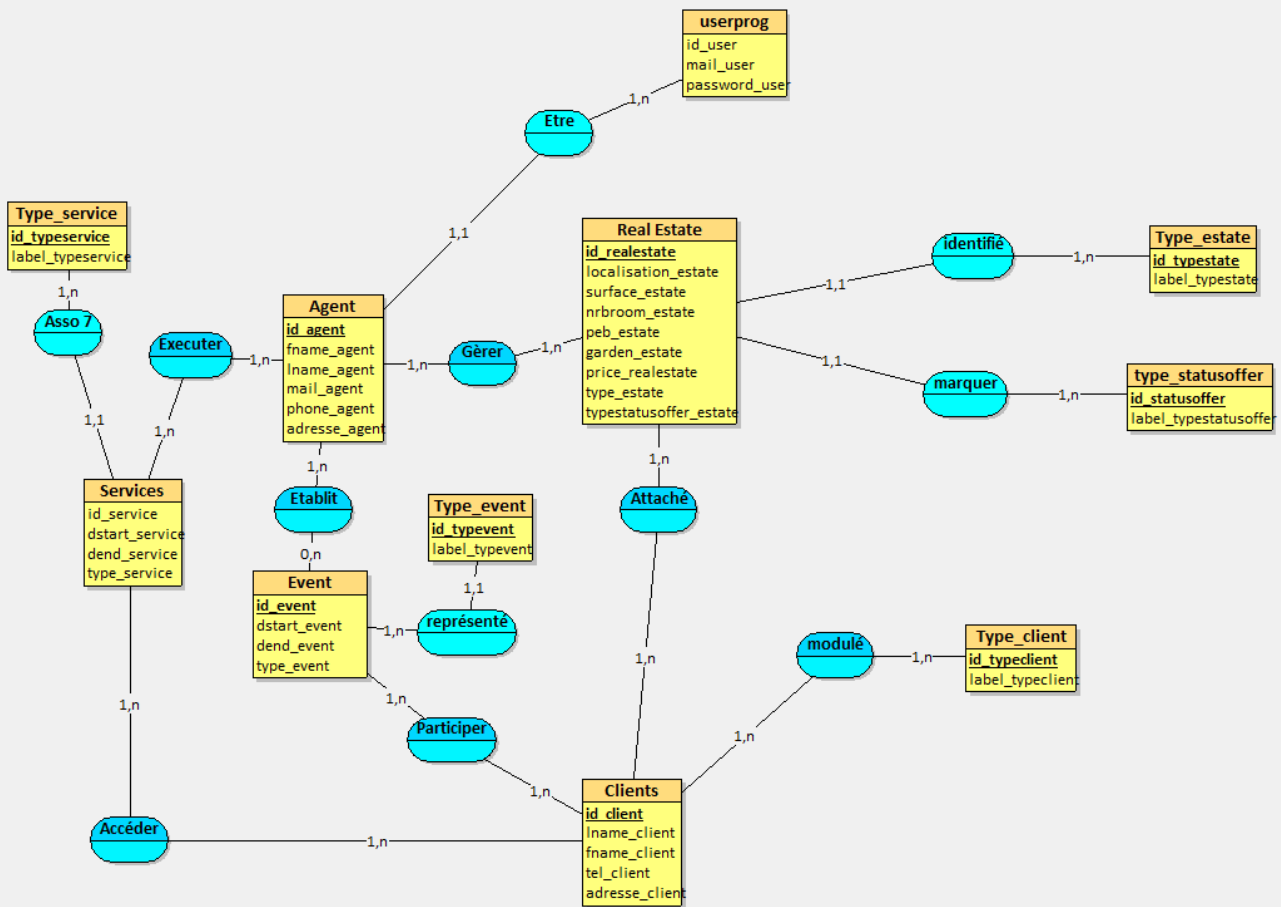
Fonctionnalités principales

- Gestion des biens immobiliers (CRUD)
- Gestion des photos et documents par bien
- Suivi des statuts avec historique (Disponible, En visite, Sous offre, Vendu, Loue, A louer)
- Gestion des evenements lies aux biens (visites, appels, accords, signatures)
- Gestion des clients (CRUD complet avec recherche et filtres)
- Gestion des agents
- Dashboard avec statistiques et calendrier hebdomadaire
- Calendrier mensuel avec navigation
- Filtres avancés sur les biens (type, statut, prix, surface, ville)

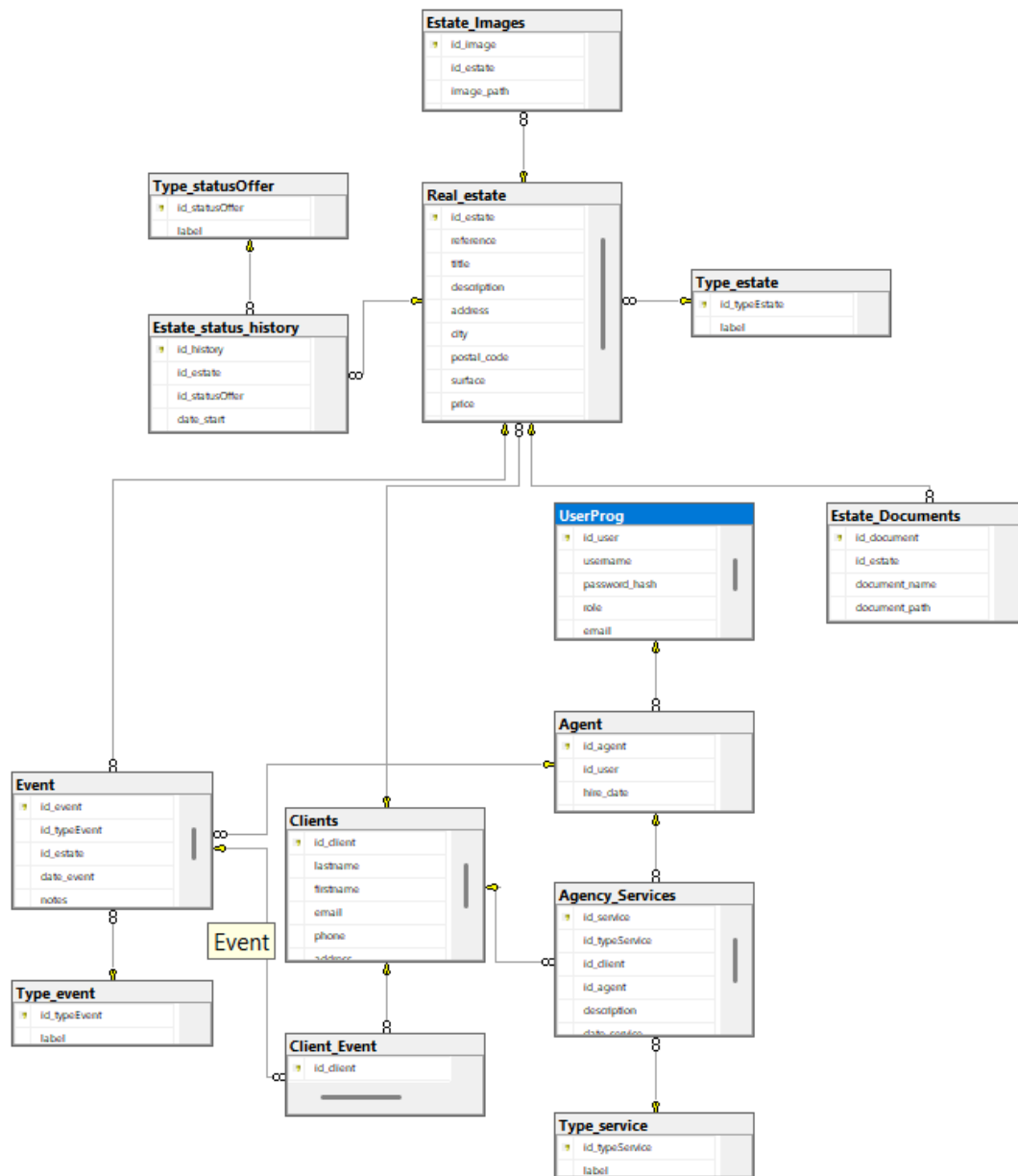
Entités de données principales

Entité	Description
RealEstate	Biens immobiliers gérés par l'agence
Client	Personnes interagissant avec l'agence
Agent	Collaborateurs de l'agence
UserProg	Comptes utilisateurs pour l'utilisation de l'app
Event	Evenements lies aux biens
EstateStatusHistory	Historique des statuts d'un bien
EstateImage	Photos des biens
EstateDocument	Documents liés aux biens
TypeEstate	Types de biens (Maison, Appartement, Terrain)
TypeStatusOffer	Types de statuts (Disponible, Vendu, etc.)
TypeEvent	Types d'événements (Visite, Appel, etc.)

3. Modèle entité-association



4. Schéma physique de la base de données

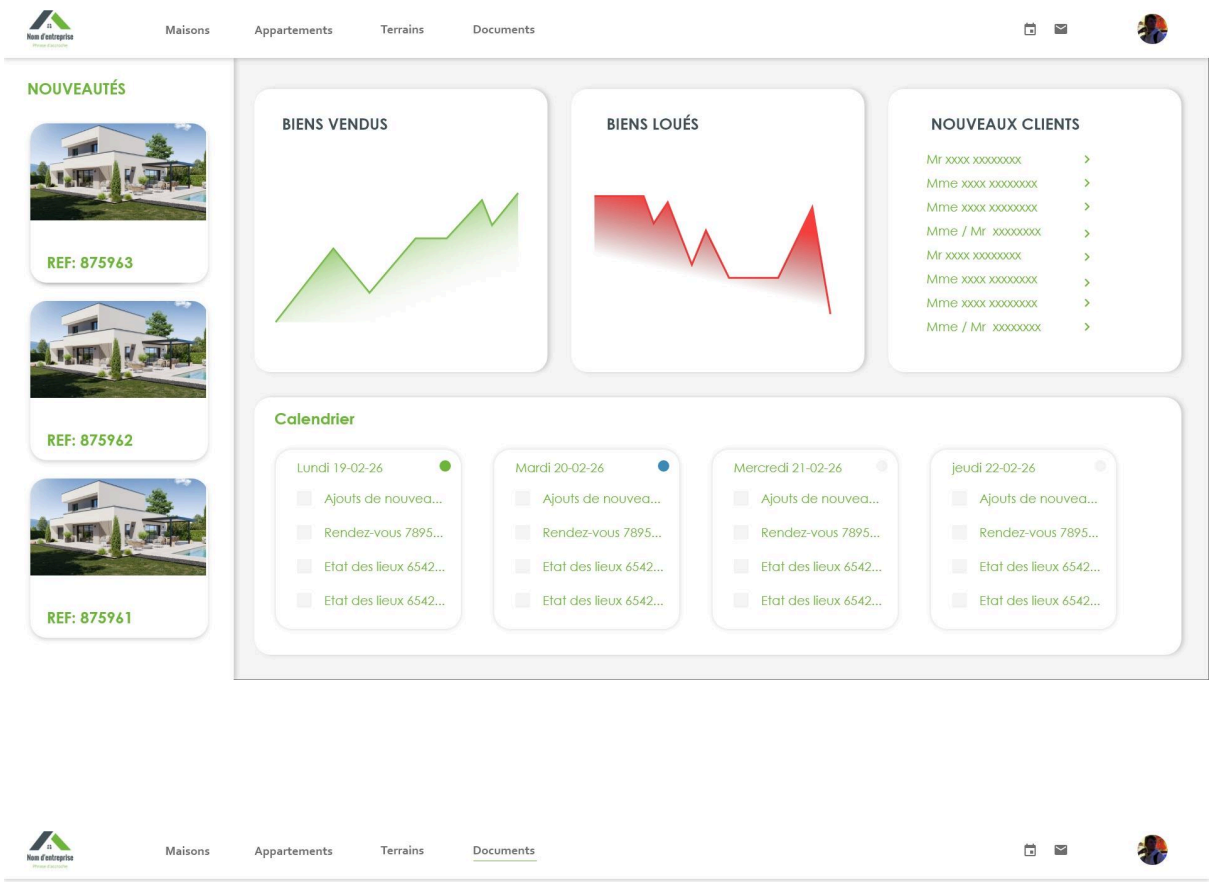


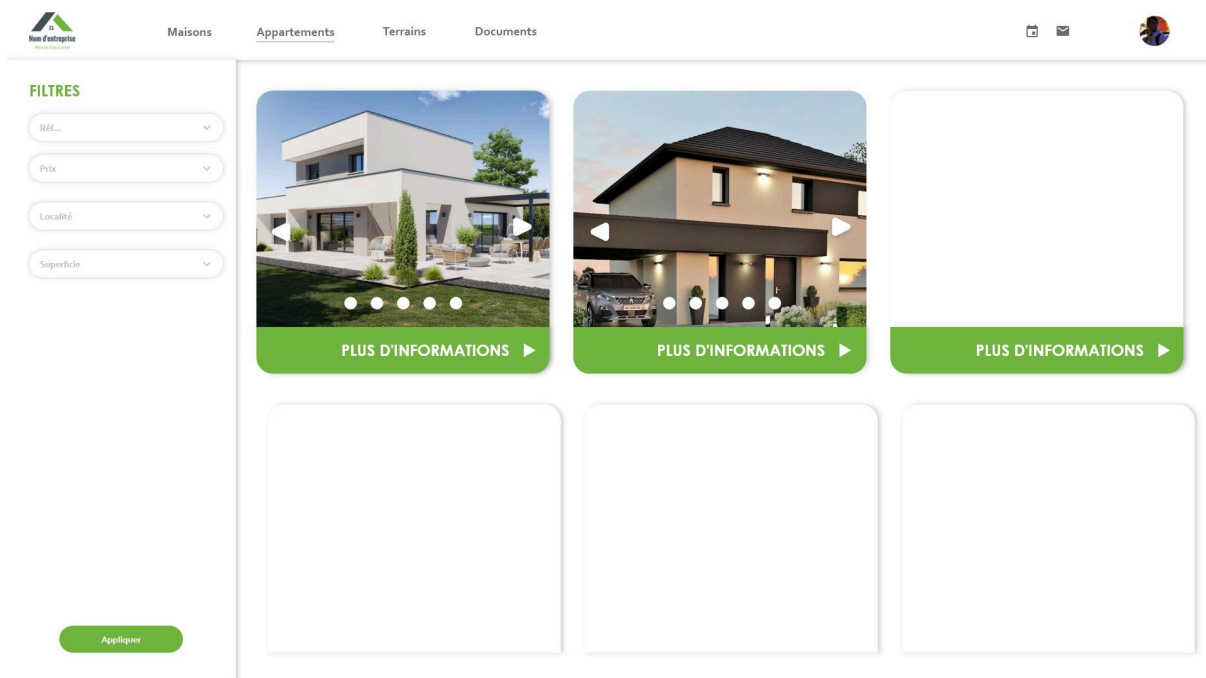
Le schéma physique comprend les tables suivantes avec leurs clés primaires, étrangères et contraintes :

- UserProg (id_user PK, username UNIQUE, password_hash, role CHECK, email, phone)
- Agent (id_agent PK, id_user FK, hire_date, active DEFAULT 1)
- Clients (id_client PK, firstname, lastname, email, phone, address, type_client CHECK)
- Real_estate (id_estate PK, reference UNIQUE, title, description, address, city, postal_code, surface, price, creation_date, id_typeEstate FK, id_owner FK)
- Type_estate (id_typeEstate PK, label UNIQUE)
- Type_statusOffer (id_statusOffer PK, label UNIQUE)

- Estate_status_history (id_history PK, id_estate FK, id_statusOffer FK, date_start, date_end)
- Type_event (id_typeEvent PK, label UNIQUE)
- Event (id_event PK, id_typeEvent FK, id_estate FK, id_agent FK, date_event, notes)
- Client_Event (id_client FK, id_event FK, role_in_event) cle primaire composite
- Estate_Images (id_image PK, id_estate FK, image_path, is_main DEFAULT 0)
- Estate_Documents (id_document PK, id_estate FK, document_name, document_path, upload_date)
- Agency_Services (id_service PK, id_typeService FK, id_client FK, id_agent FK, description, date_service, price)
- Type_service (id_typeService PK, label UNIQUE)

5. Prototypes des écrans





Les écrans principaux de l'application sont :

- Dashboard Vue d'ensemble avec nouveaux biens, statistiques et calendrier hebdomadaire
- Page Biens, Grille de cartes avec filtres avancés
- Détail d'un bien, Photos, informations, statuts, événements et documents
- Gestion des photos, Ajout, suppression et photo principale
- Liste des clients, Tableau avec recherche et CRUD complet
- Liste des agents, Gestion des agents
- Calendrier mensuel, Vue mensuelle avec navigation
- Documents, Liste de tous les documents

6. Bases de la solution .NET

Architecture en couches

La solution EasyImmo suit une architecture multicouche séparant les responsabilités :

Couche	Projet	Responsabilité
Présentation	ImmoAppUI (MAUI)	Interface utilisateur, Views, ViewModels
Métier	ImmoApp.BLL	Logique métier, validations, services
Accès données	ImmoApp.DataAccess	Modeles EF, DbContext, SQL Server
Tests	ImmoApp.TestConsole	Tests des services en ligne de commande

Pattern MVVM

L'interface utilisateur suit le patron ModelViewViewModel (MVVM) qui permet une séparation claire entre la logique et l'affichage.

- BaseViewModel implémentant INotifyPropertyChanged avec SetProperty pour la notification automatique
- ObservableCollection pour les listes avec mise à jour automatique de l'UI
- Data Binding en XAML reliant les propriétés des ViewModels aux éléments visuels
- Aucune logique métier dans les Views uniquement des appels aux ViewModels

Entity Framework

L'accès aux données est géré via Entity Framework Core avec les principes suivants :

- Modèles générés par scaffolding à partir de la base de données existante
- Utilisation du pattern using pour ouvrir et fermer la connexion DB automatiquement
- Include pour charger les relations (jointures)
- Pas d'injection de dépendance instanciation directe du DbContext dans chaque méthode

7. Structure de la solution .NET

Organisation des projets

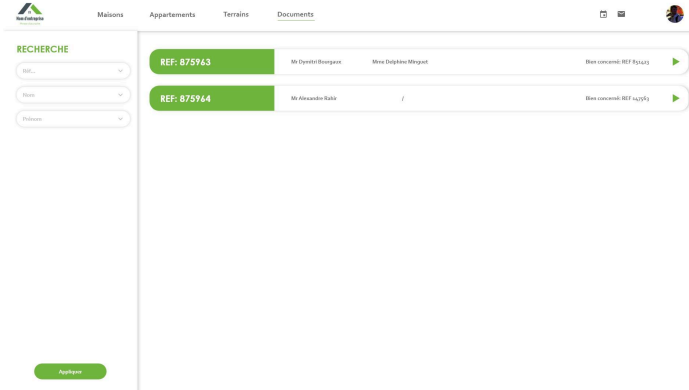
Project EasyImmo/	
├─ ImmoApp.DataAccess	→ Couche acces aux donnees
│ └─ Models/	→ Modèles EF générés par scaffold
├─ ImmoApp.BLL	→ Couche métier
│ └─ Services/	→ Services métier
├─ ImmoAppUI	→ Couche présentation (MAUI)
│ └─ Views/	→ Pages XAML
│ └─ ViewModels/	→ ViewModels
└─ ImmoApp.TestConsole	→ Tests

ImmoApp.DataAccess (Modèles)

Fichier	Description
ImmoDbContext.cs	DbContext Entity Framework connexion SQL Server
Agent.cs	Modèle agent de l'agence
Client.cs	Modèle client
RealEstate.cs	Modèle bien immobilier
Event.cs	Modele événement
EstateStatusHistory.cs	Historique des statuts
EstateImage.cs	Photos des biens
EstateDocument.cs	Documents des biens
UserProg.cs	Comptes utilisateurs
TypeEstate.cs / TypeEvent.cs / ...	Tables de référence

ImmoApp.BLL (Services)

Service	Fonctionnalites
UserService	Création d'utilisateurs
AgentService	CRUD agents + UpdateAgent
ClientService	CRUD clients + recherche
RealEstateService	CRUD biens + filtres + statistiques
EstateStatusService	Gestion statuts + historique
EstateImageService	CRUD photos + photo principale
EstateDocumentService	CRUD documents
EventService	CRUD événements + liaison clients/agents

TypeEstateService	Liste types de biens 
TypeEventService	Liste types d'événements
TypeStatusOfferService	Liste types de statuts

ImmoAppUI (ViewModels)

ViewModel	Role
BaseViewModel	Classe de base INotifyPropertyChanged + SetProperty
DashboardViewModel	Stats, derniers biens, clients, calendrier hebdo
RealEstateViewModel	Liste biens avec filtres
RealEstateDetailViewModel	Détail bien, photos, statuts, événements
ClientViewModel	CRUD clients + recherche
AgentViewModel	CRUD agents
CalendarViewModel	Calendrier mensuel avec navigation

ImmoAppUI (Views)

Page	Description
DashboardPage	Tableau de bord principal
BiensPage	Liste des biens avec filtres avancés
RealEstateDetailPage	Fiche complète d'un bien
AddRealEstatePage / EditRealEstatePage	Formulaires ajout/modification bien
ManagePhotosPage	Gestion des photos d'un bien
ManageDocumentsPage	Gestion des documents d'un bien
ClientsListPage	Liste clients avec tableau et recherche
AddClientPage / EditClientPage	Formulaires ajout/modification client
AgentsListPage	Liste des agents

AddAgentPage / EditAgentPage	Formulaires ajout/modification agent
CalendarPage	Calendrier mensuel avec evenements
DocumentPage	Menu gestion (Agents, Clients, Documents)
AllDocumentsPage	Tous les documents de tous les biens
AddEventPage	Formulaire ajout événement

8. Script SQL de création de la base de données

Le script SQL de création de la structure complète de la base de données est fourni en annexe dans le fichier script20260530.sql.

Ce script a été généré via SQL Server Management Studio et est compatible avec SQL Server 2012+. Il comprend :

- La création de toutes les tables avec leurs clés primaires
- Les contraintes de clés étrangères entre les tables
- Les contraintes CHECK (type client, rôle utilisateur)
- Les valeurs DEFAULT (dates, statuts actifs)
- Les index UNIQUE (username, references, labels)

Pour recréer la base de données complète avec les données de test, utiliser le dump SQL fourni en annexe qui contient également tous les INSERT INTO avec les 50 clients, 50 biens, 10 agents et les événements associés.

Ci dessous le script pour générer la db sans les 50 différentes données.

```
CREATE DATABASE [EsayImmo]

CONTAINMENT = NONE

ON PRIMARY

( NAME = N'EsayImmo', FILENAME = N'C:\Program Files\Microsoft SQL
Server\MSSQL16.SQLEXPRESS\MSSQL\DATA\EsayImmo.mdf' , SIZE = 8192KB ,
MAXSIZE = UNLIMITED, FILEGROWTH = 65536KB )

LOG ON

( NAME = N'EsayImmo_log', FILENAME = N'C:\Program Files\Microsoft SQL
Server\MSSQL16.SQLEXPRESS\MSSQL\DATA\EsayImmo_log.ldf' , SIZE = 8192KB
, MAXSIZE = 2048GB , FILEGROWTH = 65536KB )

GO

IF (1 = FULLTEXTSERVICEPROPERTY('IsFullTextInstalled'))
begin
EXEC [EsayImmo].[dbo].[sp_fulltext_database] @action = 'enable'
end

GO

ALTER DATABASE [EsayImmo] SET ANSI_NULL_DEFAULT OFF

GO

ALTER DATABASE [EsayImmo] SET ANSI_NULLS OFF

GO
```

```
ALTER DATABASE [EsayImmo] SET ANSI_PADDING OFF
GO

ALTER DATABASE [EsayImmo] SET ANSI_WARNINGS OFF
GO

ALTER DATABASE [EsayImmo] SET ARITHABORT OFF
GO

ALTER DATABASE [EsayImmo] SET AUTO_CLOSE OFF
GO

ALTER DATABASE [EsayImmo] SET AUTO_SHRINK OFF
GO

ALTER DATABASE [EsayImmo] SET AUTO_UPDATE_STATISTICS ON
GO

ALTER DATABASE [EsayImmo] SET CURSOR_CLOSE_ON_COMMIT OFF
GO

ALTER DATABASE [EsayImmo] SET CURSOR_DEFAULT GLOBAL
GO

ALTER DATABASE [EsayImmo] SET CONCAT_NULL_YIELDS_NULL OFF
GO

ALTER DATABASE [EsayImmo] SET NUMERIC_ROUNDABORT OFF
GO

ALTER DATABASE [EsayImmo] SET QUOTED_IDENTIFIER OFF
GO

ALTER DATABASE [EsayImmo] SET RECURSIVE_TRIGGERS OFF
GO

ALTER DATABASE [EsayImmo] SET  DISABLE_BROKER
GO

ALTER DATABASE [EsayImmo] SET AUTO_UPDATE_STATISTICS_ASYNC OFF
GO

ALTER DATABASE [EsayImmo] SET DATE_CORRELATION_OPTIMIZATION OFF
GO

ALTER DATABASE [EsayImmo] SET TRUSTWORTHY OFF
```

```
GO

ALTER DATABASE [EsayImmo] SET ALLOW_SNAPSHOT_ISOLATION OFF

GO

ALTER DATABASE [EsayImmo] SET PARAMETERIZATION SIMPLE

GO

ALTER DATABASE [EsayImmo] SET READ_COMMITTED_SNAPSHOT OFF

GO

ALTER DATABASE [EsayImmo] SET HONOR_BROKER_PRIORITY OFF

GO

ALTER DATABASE [EsayImmo] SET RECOVERY SIMPLE

GO

ALTER DATABASE [EsayImmo] SET MULTI_USER

GO

ALTER DATABASE [EsayImmo] SET PAGE_VERIFY CHECKSUM

GO

ALTER DATABASE [EsayImmo] SET DB_CHAINING OFF

GO

ALTER DATABASE [EsayImmo] SET FILESTREAM( NON_TRANSACTED_ACCESS = OFF )

GO

ALTER DATABASE [EsayImmo] SET TARGET_RECOVERY_TIME = 60 SECONDS

GO

USE [EsayImmo]

GO

/***** Object:  Table [dbo].[Agency_Services]      Script Date: 30-05-26
23:18:19 *****/

SET ANSI_NULLS ON

GO

SET QUOTED_IDENTIFIER ON

GO

CREATE TABLE [dbo].[Agency_Services](
    [id_service] [int] IDENTITY(1,1) NOT NULL,
    [id_typeService] [int] NOT NULL,
```



```

        [id_client] [int] NOT NULL,

        [id_agent] [int] NULL,

        [description] [nvarchar](max) NULL,

        [date_service] [datetime] NULL,

        [price] [decimal](12, 2) NULL,

PRIMARY KEY CLUSTERED
(
        [id_service] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY =
OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY]
) ON [PRIMARY] TEXTIMAGE_ON [PRIMARY]
GO

/***** Object:  Table [dbo].[Agent]      Script Date: 30-05-26 23:18:19
*****/

SET ANSI_NULLS ON
GO

SET QUOTED_IDENTIFIER ON
GO

CREATE TABLE [dbo].[Agent] (
        [id_agent] [int] IDENTITY(1,1) NOT NULL,

        [id_user] [int] NOT NULL,

        [hire_date] [date] NULL,

        [active] [bit] NULL,

PRIMARY KEY CLUSTERED
(
        [id_agent] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY =
OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY]
) ON [PRIMARY]
GO

/***** Object:  Table [dbo].[Client_Event]      Script Date: 30-05-26
23:18:19 *****/

SET ANSI_NULLS ON

```

```

GO

SET QUOTED_IDENTIFIER ON

GO

CREATE TABLE [dbo].[Client_Event] (
    [id_client] [int] NOT NULL,
    [id_event] [int] NOT NULL,
    [role_in_event] [nvarchar](50) NULL,
PRIMARY KEY CLUSTERED
(
    [id_client] ASC,
    [id_event] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY =
OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY]
) ON [PRIMARY]

GO

/***** Object:  Table [dbo].[Clients]      Script Date: 30-05-26
23:18:19 *****/

SET ANSI_NULLS ON

GO

SET QUOTED_IDENTIFIER ON

GO

CREATE TABLE [dbo].[Clients] (
    [id_client] [int] IDENTITY(1,1) NOT NULL,
    [lastname] [nvarchar](100) NOT NULL,
    [firstname] [nvarchar](100) NOT NULL,
    [email] [nvarchar](150) NULL,
    [phone] [nvarchar](20) NULL,
    [address] [nvarchar](255) NULL,
    [type_client] [nvarchar](50) NULL,
PRIMARY KEY CLUSTERED
(
    [id_client] ASC

```

```

)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY =
OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY]
) ON [PRIMARY]
GO

/***** Object: Table [dbo].[Estate_Documents]    Script Date:
30-05-26 23:18:19 *****/

SET ANSI_NULLS ON
GO

SET QUOTED_IDENTIFIER ON
GO

CREATE TABLE [dbo].[Estate_Documents](
    [id_document] [int] IDENTITY(1,1) NOT NULL,
    [id_estate] [int] NULL,
    [document_name] [nvarchar](255) NULL,
    [document_path] [nvarchar](500) NULL,
    [upload_date] [datetime] NULL,
PRIMARY KEY CLUSTERED
(
    [id_document] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY =
OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY]
) ON [PRIMARY]
GO

/***** Object: Table [dbo].[Estate_Images]    Script Date: 30-05-26
23:18:19 *****/

SET ANSI_NULLS ON
GO

SET QUOTED_IDENTIFIER ON
GO

CREATE TABLE [dbo].[Estate_Images](
    [id_image] [int] IDENTITY(1,1) NOT NULL,
    [id_estate] [int] NULL,
    [image_path] [nvarchar](500) NULL,

```

```

        [is_main] [bit] NULL,
PRIMARY KEY CLUSTERED
(
    [id_image] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY =
OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY]
) ON [PRIMARY]
GO

/***** Object:  Table [dbo].[Estate_status_history]    Script Date:
30-05-26 23:18:19 *****/

SET ANSI_NULLS ON
GO

SET QUOTED_IDENTIFIER ON
GO

CREATE TABLE [dbo].[Estate_status_history](
    [id_history] [int] IDENTITY(1,1) NOT NULL,
    [id_estate] [int] NOT NULL,
    [id_statusOffer] [int] NOT NULL,
    [date_start] [datetime] NOT NULL,
    [date_end] [datetime] NULL,
PRIMARY KEY CLUSTERED
(
    [id_history] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY =
OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY]
) ON [PRIMARY]
GO

/***** Object:  Table [dbo].[Event]    Script Date: 30-05-26 23:18:19
*****/

SET ANSI_NULLS ON
GO

SET QUOTED_IDENTIFIER ON
GO

```

```

CREATE TABLE [dbo].[Event](
    [id_event] [int] IDENTITY(1,1) NOT NULL,
    [id_typeEvent] [int] NOT NULL,
    [id_estate] [int] NOT NULL,
    [date_event] [datetime] NOT NULL,
    [notes] [nvarchar](max) NULL,
    [id_agent] [int] NULL,
PRIMARY KEY CLUSTERED
(
    [id_event] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY =
OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY]
) ON [PRIMARY] TEXTIMAGE_ON [PRIMARY]
GO

/***** Object:  Table [dbo].[Real_estate]      Script Date: 30-05-26
23:18:19 *****/

SET ANSI_NULLS ON
GO

SET QUOTED_IDENTIFIER ON
GO

CREATE TABLE [dbo].[Real_estate](
    [id_estate] [int] IDENTITY(1,1) NOT NULL,
    [reference] [nvarchar](50) NOT NULL,
    [title] [nvarchar](150) NOT NULL,
    [description] [nvarchar](max) NULL,
    [address] [nvarchar](255) NOT NULL,
    [city] [nvarchar](100) NULL,
    [postal_code] [nvarchar](10) NULL,
    [surface] [decimal](10, 2) NULL,
    [price] [decimal](12, 2) NULL,
    [creation_date] [datetime] NULL,
    [id_typeEstate] [int] NOT NULL,

```

```

        [id_owner] [int] NULL,
PRIMARY KEY CLUSTERED
(
        [id_estate] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY =
OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY],
UNIQUE NONCLUSTERED
(
        [reference] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY =
OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY]
) ON [PRIMARY] TEXTIMAGE_ON [PRIMARY]
GO

/***** Object:  Table [dbo].[Type_estate]      Script Date: 30-05-26
23:18:19 *****/

SET ANSI_NULLS ON
GO

SET QUOTED_IDENTIFIER ON
GO

CREATE TABLE [dbo].[Type_estate] (
        [id_typeEstate] [int] IDENTITY(1,1) NOT NULL,
        [label] [nvarchar](50) NOT NULL,
PRIMARY KEY CLUSTERED
(
        [id_typeEstate] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY =
OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY],
UNIQUE NONCLUSTERED
(
        [label] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY =
OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY]
) ON [PRIMARY]

```

```

GO

/***** Object:  Table [dbo].[Type_event]      Script Date: 30-05-26
23:18:19 *****/

SET ANSI_NULLS ON

GO

SET QUOTED_IDENTIFIER ON

GO

CREATE TABLE [dbo].[Type_event] (
    [id_typeEvent] [int] IDENTITY(1,1) NOT NULL,
    [label] [nvarchar](50) NOT NULL,
PRIMARY KEY CLUSTERED
(
    [id_typeEvent] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY =
OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY],
UNIQUE NONCLUSTERED
(
    [label] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY =
OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY]
) ON [PRIMARY]

GO

/***** Object:  Table [dbo].[Type_service]      Script Date: 30-05-26
23:18:19 *****/

SET ANSI_NULLS ON

GO

SET QUOTED_IDENTIFIER ON

GO

CREATE TABLE [dbo].[Type_service] (
    [id_typeService] [int] IDENTITY(1,1) NOT NULL,
    [label] [nvarchar](50) NOT NULL,
PRIMARY KEY CLUSTERED
(

```

```

        [id_typeService] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY =
OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY],
UNIQUE NONCLUSTERED
(
        [label] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY =
OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY]
) ON [PRIMARY]
GO

/***** Object:  Table [dbo].[Type_statusOffer]      Script Date:
30-05-26 23:18:19 *****/

SET ANSI_NULLS ON
GO

SET QUOTED_IDENTIFIER ON
GO

CREATE TABLE [dbo].[Type_statusOffer](
        [id_statusOffer] [int] IDENTITY(1,1) NOT NULL,
        [label] [nvarchar](50) NOT NULL,
PRIMARY KEY CLUSTERED
(
        [id_statusOffer] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY =
OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY],
UNIQUE NONCLUSTERED
(
        [label] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY =
OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY]
) ON [PRIMARY]
GO

/***** Object:  Table [dbo].[UserProg]      Script Date: 30-05-26
23:18:19 *****/

```



```
SET ANSI_NULLS ON

GO

SET QUOTED_IDENTIFIER ON

GO

CREATE TABLE [dbo].[UserProg] (

    [id_user] [int] IDENTITY(1,1) NOT NULL,

    [username] [nvarchar](50) NOT NULL,

    [password_hash] [nvarchar](255) NOT NULL,

    [role] [nvarchar](50) NULL,

    [email] [nvarchar](150) NULL,

    [phone] [nvarchar](20) NULL,

PRIMARY KEY CLUSTERED

(

    [id_user] ASC

)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY],

UNIQUE NONCLUSTERED

(

    [username] ASC

)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY]

) ON [PRIMARY]

GO

ALTER TABLE [dbo].[Agency_Services] ADD DEFAULT (getdate()) FOR [date_service]

GO

ALTER TABLE [dbo].[Agent] ADD DEFAULT ((1)) FOR [active]

GO

ALTER TABLE [dbo].[Estate_Documents] ADD DEFAULT (getdate()) FOR [upload_date]

GO

ALTER TABLE [dbo].[Estate_Images] ADD DEFAULT ((0)) FOR [is_main]

GO
```

```
ALTER TABLE [dbo].[Real_estate] ADD DEFAULT (getdate()) FOR
[creation_date]

GO

ALTER TABLE [dbo].[Agency_Services] WITH CHECK ADD FOREIGN
KEY([id_agent])

REFERENCES [dbo].[Agent] ([id_agent])

GO

ALTER TABLE [dbo].[Agency_Services] WITH CHECK ADD FOREIGN
KEY([id_client])

REFERENCES [dbo].[Clients] ([id_client])

GO

ALTER TABLE [dbo].[Agency_Services] WITH CHECK ADD FOREIGN
KEY([id_typeService])

REFERENCES [dbo].[Type_service] ([id_typeService])

GO

ALTER TABLE [dbo].[Agent] WITH CHECK ADD FOREIGN KEY([id_user])

REFERENCES [dbo].[UserProg] ([id_user])

GO

ALTER TABLE [dbo].[Client_Event] WITH CHECK ADD FOREIGN
KEY([id_client])

REFERENCES [dbo].[Clients] ([id_client])

GO

ALTER TABLE [dbo].[Client_Event] WITH CHECK ADD FOREIGN
KEY([id_event])

REFERENCES [dbo].[Event] ([id_event])

GO

ALTER TABLE [dbo].[Estate_Documents] WITH CHECK ADD FOREIGN
KEY([id_estate])

REFERENCES [dbo].[Real_estate] ([id_estate])

GO

ALTER TABLE [dbo].[Estate_Images] WITH CHECK ADD FOREIGN
KEY([id_estate])

REFERENCES [dbo].[Real_estate] ([id_estate])

GO
```

```

ALTER TABLE [dbo].[Estate_status_history] WITH CHECK ADD FOREIGN
KEY([id_estate])

REFERENCES [dbo].[Real_estate] ([id_estate])

GO

ALTER TABLE [dbo].[Estate_status_history] WITH CHECK ADD FOREIGN
KEY([id_statusOffer])

REFERENCES [dbo].[Type_statusOffer] ([id_statusOffer])

GO

ALTER TABLE [dbo].[Event] WITH CHECK ADD FOREIGN KEY([id_agent])

REFERENCES [dbo].[Agent] ([id_agent])

GO

ALTER TABLE [dbo].[Event] WITH CHECK ADD FOREIGN KEY([id_estate])

REFERENCES [dbo].[Real_estate] ([id_estate])

GO

ALTER TABLE [dbo].[Event] WITH CHECK ADD FOREIGN KEY([id_typeEvent])

REFERENCES [dbo].[Type_event] ([id_typeEvent])

GO

ALTER TABLE [dbo].[Real_estate] WITH CHECK ADD FOREIGN KEY([id_owner])

REFERENCES [dbo].[Clients] ([id_client])

GO

ALTER TABLE [dbo].[Real_estate] WITH CHECK ADD FOREIGN
KEY([id_typeEstate])

REFERENCES [dbo].[Type_estate] ([id_typeEstate])

GO

ALTER TABLE [dbo].[Clients] WITH CHECK ADD CHECK
((([type_client]='Partenaire' OR [type_client]='Locataire' OR
[type_client]='Acheteur' OR [type_client]='Propriétaire'))

GO

ALTER TABLE [dbo].[UserProg] WITH CHECK ADD CHECK
(((role]='Assistant' OR [role]='Agent' OR [role]='Admin'))

GO

USE [master]

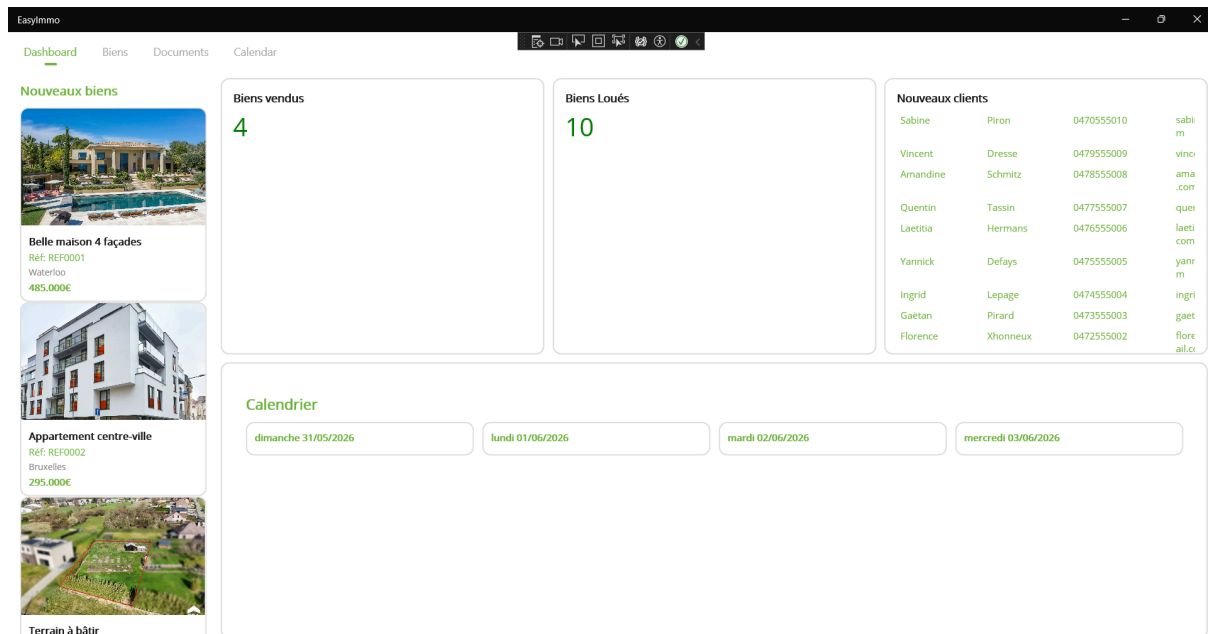
GO

```

```
ALTER DATABASE [EssayImmo] SET READ_WRITE  
GO
```

9. Présentation de l'application

Dashboard



Le dashboard est la page principale de l'application. Il se présente avec:

- Une colonne de gauche avec les 5 derniers biens ajoutés avec leur photo et référence (utilisation du scroll pour voir les deux autres biens)
- Trois cartes de statistiques : biens vendus, biens loués (avec indicateur couleur), nouveaux clients
- Un calendrier hebdomadaire affichant les événements des 4 prochains jours

Page Biens



La page biens présente les biens sous forme de cartes avec photo. Elle intègre :

- Filtres avancés : type de bien, statut, référence, prix maximum, localité, surface minimum
- Affichage en grille 3 colonnes avec photo principale et bouton de détail
- Bouton FAB (+) en bas à droite pour ajouter un nouveau bien

Détail d'un bien

Dashboard
Propri t 
Documents
Calendrier

G rer les photos

Informations
R f: 000009
Prix: 452.000 
Surface: 620.00m 
Avenue du test de la maison
Test Maison
Text pour la cr ation d'une maison

Propri taire
Fr d ric
Parisien

Statut actuel : Vendu
Changer le statut

Appliquer

Historique	
Vendu	24-05-2026
Sous offre	18-05-2026
Disponible	15-05-2026

 v nements

+ Ajouter

Date	Description	Auteur	Action
24-05-2026	test	admin	Supprimer

Mettre   jour

Supporter

La page de détail d'un bien centralise toutes les informations :

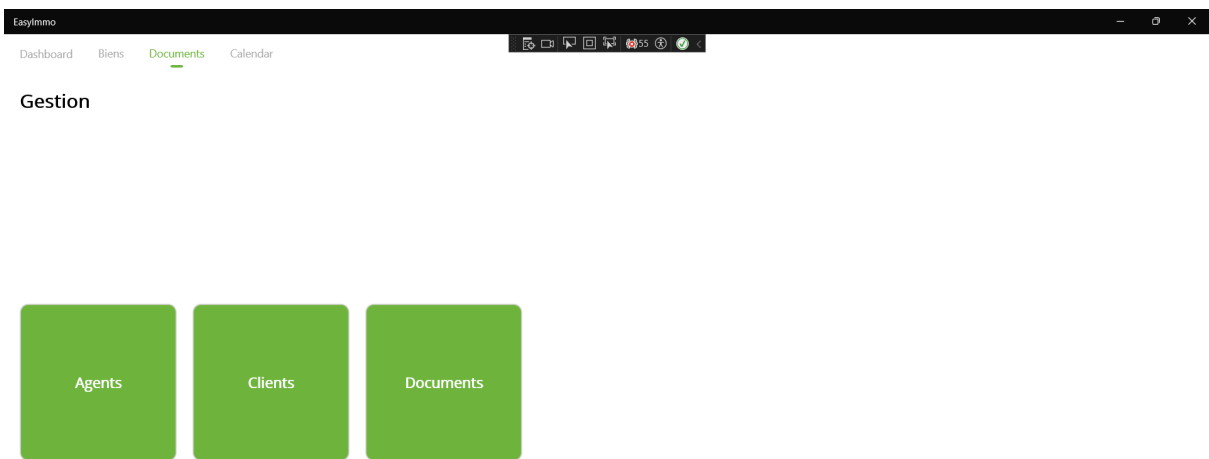
- Galerie photo horizontale avec gestion des images
- Carte d'informations (référence, prix, surface, adresse)
- Carte propriétaire avec coordonnées
- Carte documents avec accès aux fichiers
- Gestion des statuts avec historique complet
- Liste des événements avec ajout et suppression
- Boutons modification et suppression du bien

Calendrier

Le calendrier mensuel permet de visualiser tous les événements du mois. Il offre :

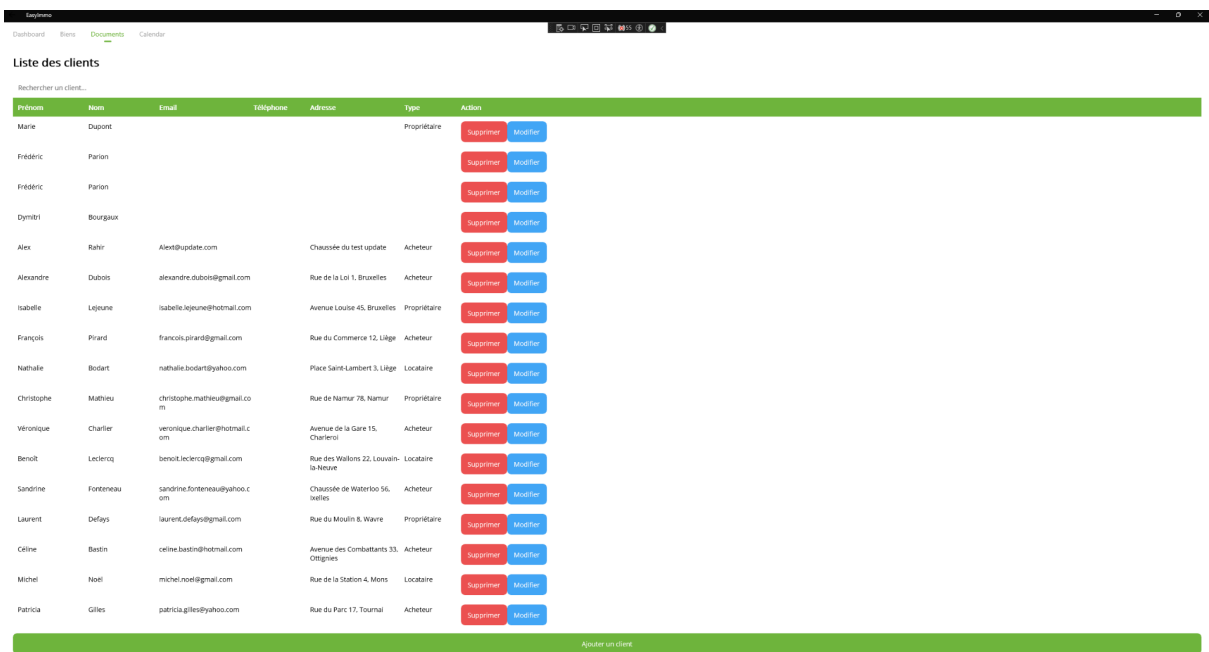
- Navigation entre les mois avec boutons précédent/suivant
- Affichage mensuel en grille 7 colonnes (lundi au dimanche)
- Evénements cliquables redirigeant vers le bien concerné
- Affichage du type d'événement, de la référence du bien et de l'agent responsable

Documentations de l'application



La page Document regroupe l'endroit où vous trouverez les listes des clients, des agents et aussi l'entièreté des documents qui sont liés à des biens.

Gestion des clients



La gestion des clients propose un tableau complet avec :

- Barre de recherche en temps réel
- Affichage en tableau avec toutes les informations
- Boutons modifier et supprimer sur chaque ligne

- Formulaire d'ajout et de modification

Gestion des agents

Tous les agents

DashboardsItemsDocumentsCalendar

admin 01/01/2024	ModifierSupprimer
Professor X 01/01/2026	ModifierSupprimer
jean.dupont 15/01/2020	ModifierSupprimer
mario.lambert 20/01/19	ModifierSupprimer
pierre.martin 01/06/2021	ModifierSupprimer
sophie.lecomte 10/09/2018	ModifierSupprimer
thomas.henry 14/02/2022	ModifierSupprimer
jolie.simon 20/07/2020	ModifierSupprimer
nicolas.adam 05/11/2017	ModifierSupprimer
laura.renard 09/01/2023	ModifierSupprimer
david.colin 22/06/2019	ModifierSupprimer
emma.gerard 17/04/2021	ModifierSupprimer

+ Ajouter un agent

La gestion des agents propose un tableau complet avec :

- Affichage en tableau avec toutes les informations
- Boutons modifier et supprimer sur chaque ligne

Formulaire d'ajout et de modification

10. Conclusion

Le projet fut assez complexe car j'ai beaucoup de difficultés avec la logique C#, même si j'apprécie beaucoup ce langage de programmation. Ce fut différent pour MAUI où, étant développeur web et fortement habitué au CSS, je n'ai jamais vraiment réussi à trouver mes marques, ce qui se ressent dans le design de mon application.

En revanche, ce que j'ai particulièrement apprécié et qui constituait une nouvelle découverte pour moi, c'était Entity Framework. Gérer les données directement depuis le code fut relativement simple grâce à ce framework.

J'ai commencé le projet fin mars et je pense avoir correctement géré mon temps. Certaines fonctions et bibliothèques furent assez difficiles à trouver et à implémenter, ce qui me frustra de ne pas avoir pu réaliser 100% de mes idées initiales, comme par exemple les statistiques et graphiques. Je reste néanmoins satisfait car je pense avoir bien répondu aux demandes de base, dans le temps imparti et même légèrement en avance.

Si je devais refaire le projet, je travaillerais davantage la conception de ma base de données car, lors du développement, j'ai dû ajouter quelques tables auxquelles je n'avais pas pensé lors de l'analyse préliminaire.

Annexes

Document d'instructions Configuration

Prérequis

- Visual Studio 2022 ou supérieur
- SQL Server Express 2019 ou supérieur
- SQL Server Management Studio (SSMS)
- .NET 8 SDK

Etapes de configuration

1. Cloner le repository GitHub <https://github.com/TheLordRoukmout/EasyImmo> ou le télécharger via la section devoir.
2. Ouvrir SQL Server Management Studio et exécuter le fichier script20260530.sql
3. Ouvrir la solution Project EasyImmo.sln dans Visual Studio (.net 8)
4. Vérifier la connection string dans ImmoApp.DataAccess/Models/ImmoDbContext.cs
5. Définir ImmoAppUI comme projet de démarrage
6. Lancer l'app